01. Autobahn BFT 개요

목차

1. Autobahn BFT란?
2. 핵심 특징
3. Narwhal과의 차이점
4. 시스템 아키텍처
5. 성능 향상 원리

Autobahn BFT란?

Autobahn BFT는 Narwhal 컨센서스 프로토콜을 확장한 고성능 비잔틴 장애 허용(Byzantine Fault Tolerant) 컨센서스 알고리즘입니다. 정상적인 네트워크 조건에서 **Fast Path**를 통해 더 높은 처리량과 낮은 지연시간을 제공하며, 비잔틴 행동이 감지되거나 네트워크 조건이 악화될 때 **Slow Path**로 자동 전환됩니다.

핵심 특징

1. 이중 경로 컨센서스 (Dual-Path Consensus)

```
pub  enum ConsensusMode {

FastPath, // 빠른 경로: 정상 조건에서 사용

SlowPath, // 느린 경로: 비잔틴 행동 감지 시 사용

Transitioning, // 전환 중: 모드 변경 과정
```

```
}
```

2. 적응형 타임아웃 관리

```
pub struct TimeoutManager {
base_timeout: Duration,
current_timeout: Duration,
adaptation_factor: f64,
max_timeout: Duration,
min_timeout: Duration,
}
```

3. 메시지 집계 최적화

```
pub struct MessageAggregator {
batch_size_threshold: usize,
batch_timeout: Duration,
pending_messages: HashMap<Round, Vec<Message>>,
}
```

4. 비잔틴 행동 감지

```
impl SlowPathConsensus {
```

```
pub  async  fn  detect_byzantine_behavior(&mut  self, certificate: &Certificate)

// 1. 이중 투표 감지 (Equivocation)

// 2. 잘못된 부모 참조 감지

// 3. 라운드 진행 위반 감지

// 4. 스테이크 임계값 위반 감지

}

}
```

Narwhal과의 차이점

| 특징 | Narwhal (Bullshark) | Autobahn BFT |

|------|---------------------|---------------|

| **컨센서스 경로** | 단일 경로 | 이중 경로 (Fast/Slow) |

| **지연시간** | 표준 | 20-30% 개선 |

| **처리량** | 기준선 | 20%+ 향상 |

| **적응성** | 고정 파라미터 | 동적 적응 |

| **비잔틴 감지** | 기본 | 고급 패턴 감지 |

시스템 아키텍처

```
┌──────────────────────────────────────────────────────────────┐

│ AutobahnController │

├──────────────────────────────────────────────────────────────┤
```

```
│ ┌──────────────────┐ ┌──────────────────────┐ │
│ │ FastPath │ │ SlowPath │ │
│ │ Consensus │◄──────────────►│ (Bullshark) │ │
│ └──────────────────┘ └──────────────────────┘ │
├──────────────────────────────────────────────────────────────┤
│ ┌──────────────────┐ ┌──────────────────┐ ┌──────────────┐ │
│ │ TimeoutManager │ │ MessageAggregator│ │ Metrics │ │
│ └──────────────────┘ └──────────────────┘ └──────────────┘ │
└──────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────┐
│ Narwhal Base Layer │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────────────┐ │
│ │ Primary │ │ Workers │ │ DAG Manager │ │
│ └──────────────┘ └──────────────┘ └──────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

주요 컴포넌트

1. AutobahnController

- **역할**: Fast Path와 Slow Path 간 조정
- **주요 함수**:

```
pub  async  fn  get_current_mode(&self) -> ConsensusMode

pub  async  fn  execute_mode_transition(&mut  self, target_mode: ConsensusMode)

pub  async  fn  process_certificate(&mut  self, certificate: AutobahnCertificate
```

2. FastPathConsensus

- **역할**: 정상 조건에서의 빠른 컨센서스
- **주요 함수**:

```
pub  fn  can_fast_commit(&self, round: Round) -> bool

pub  fn  add_certificate(&mut  self, round: Round, certificate: Certificate) ->

pub  fn  handle_round_timeout(&mut  self, round: Round) -> Result<TimeoutAction,
```

3. SlowPathConsensus

- **역할**: 비잔틴 장애 허용 및 폴백 메커니즘
- **주요 함수**:

```
pub  async  fn  detect_byzantine_behavior(&mut  self, certificate: &Certificate)

pub  async  fn  trigger_automatic_fallback(&mut  self, reason: TransitionReason)
```

성능 향상 원리

1. Fast Path 최적화

- **2f+1 임계값**: 비잔틴 노드 f개에 대해 2f+1개 노드만으로 빠른 합의
- **조기 커밋**: 임계값 도달 시 즉시 커밋
- **메시지 라운드 감소**: 정상 조건에서 메시지 교환 횟수 최소화

2. 적응형 타임아웃

```
impl TimeoutManager {
pub fn record_latency(&mut self, latency: Duration) {
// 네트워크 지연시간 기록
self.network_latency_history.push_back(latency);
self.update_timeout(); // 동적 타임아웃 조정
}
fn update_timeout(&mut self) {
let avg_latency = self.calculate_average_latency();
let adaptation_multiplier = 1.0 + self.adaptation_factor * latency_ratio;
self.current_timeout = Duration::from_millis(
(self.base_timeout.as_millis() as f64 * adaptation_multiplier) as u64
);
}
}
```

3. 메시지 집계

```
impl MessageAggregator {
```

```
pub fn add_message(&mut self, message: Message) -> Result<Option<MessageBatch

// 메시지를 배치에 추가

self.pending_messages.entry(round).or_insert_with(Vec::new).push(message);

// 배치 생성 조건 확인

if self.should_create_batch(round) {

self.create_batch(round)

} else {

Ok(None)

}

}

fn should_create_batch(&self, round: Round) -> bool {

// 크기 임계값 또는 타임아웃 도달 시 배치 생성

messages.len() >= self.batch_size_threshold ||

start_time.elapsed() >= self.batch_timeout

}

}
```

4. 비잔틴 감지 및 전환

```
impl AutobahnController {

async fn should_fallback_to_slow_path(&self, metrics: &AutobahnMetrics, timeou

// 비잔틴 행동 감지
```

```
if  self.is_byzantine_detected().await {
return Ok(true);
}
// 연속 타임아웃 확인
if  timeout_stats.consecutive_timeouts >= 3 {
return Ok(true);
}
// Fast Path 성공률 확인
if  performance_metrics.fast_path_success_rate < 0.5 {
return Ok(true);
}
Ok(false)
}
}
```

성능 지표

예상 성능 향상

- **지연시간**: 20-30% 감소
- **처리량**: 20%+ 증가
- **Fast Path 사용률**: 정상 조건에서 80%+
- **네트워크 효율성**: 메시지 집계로 30% 대역폭 절약

모니터링 메트릭

```rust
pub struct AutobahnMetrics {
pub fast_path_usage_ratio: f64,
pub average_consensus_latency: Duration,
pub throughput_certificates_per_second: f64,
pub byzantine_events_detected: u64,
pub mode_transitions: Vec<(Timestamp, ConsensusMode, ConsensusMode)>,
}
```

이 개요를 통해 Autobahn BFT의 전체적인 구조와 작동 원리를 이해할 수 있습니다. 다음 문서에서는 각 컴포넌트의 상세한 동작 과정을 살펴보겠습니다.